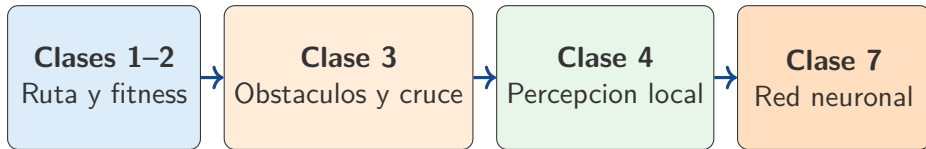


Laboratorio 7

Red neuronal pequena y evolucion por algoritmo genetico

Inteligencia Artificial



Pregunta de hoy

Podemos conservar la percepcion local y cambiar la forma en que el robot representa su politica?

Se conserva

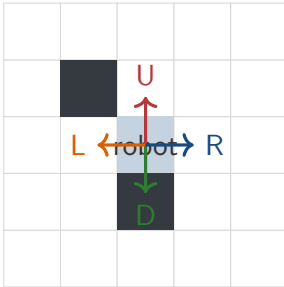
- La cuadrícula y los obstáculos.
- Los sensores U, D, L y R.
- El recorrido del robot.
- El fitness y el algoritmo evolutivo.

Cambia

- La tabla de reglas desaparece.
- Cada individuo es una red.
- Los genes son pesos numéricos.
- Se usa PyTorch para decidir.

Idea central

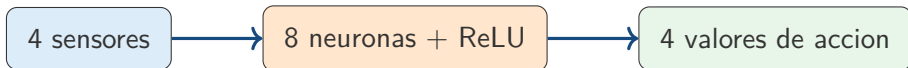
La evolución sigue buscando una buena política; ahora la política está parametrizada por una red neuronal.



- 0: celda libre.
- 1: obstaculo o borde.
- Entrada: cuatro bits.
- Salida: cuatro acciones.

Percepcion parcial

El robot no ve el tablero completo ni recibe una ruta.



Decision

La red calcula un valor para U, D, L y R. El robot ejecuta la accion con mayor valor mediante argmax .

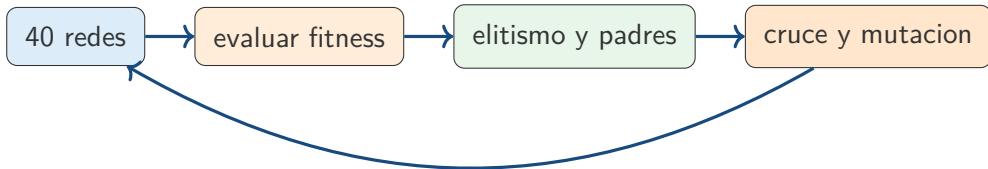
Representacion

En la politica tabular evolucionabamos reglas. Aqui evolucionamos todos los pesos de la red.

```
class RedPercepcion(nn.Module):  
    def __init__(self, hidden_size=8):  
        super().__init__()  
        self.hidden = nn.Linear(4, hidden_size)  
        self.out = nn.Linear(hidden_size, 4)  
  
    def forward(self, x):  
        return self.out(torch.relu(self.hidden(x)))
```

Lectura del codigo

Cuatro entradas representan la percepcion local; la capa oculta combina la informacion y cuatro salidas compiten por controlar el siguiente movimiento.



Fitness

$$F = 500 - 35d + 4p - 30c - 3r + 2000m$$

Se premia acercarse y llegar; se penalizan choques y repeticiones.

Durante el entrenamiento

- Se prueban muchas redes.
- Se conserva la mejor.
- La evolucion puede terminar.

Despues del entrenamiento

- Se guardan los pesos en `mejor_red.pt`.
- Se carga la misma politica.
- Se prueba en otro mapa.

Separar aprender y probar

Esto permite medir si la politica funciona solo en el mapa de entrenamiento o si responde bien a una nueva distribucion de obstaculos.

```
# Entrenar y guardar
python robot_red_neuronal_evolutiva.py \\  
    --generaciones 40 --guardar-mejor mejor_red.pt  
  
# Cargar sin volver a evolucionar  
python robot_red_neuronal_evolutiva.py \\  
    --cargar-red mejor_red.pt
```

Mapa alternativo

Se modifica manualmente OBSTACULOS, INICIO o META en el archivo y se repite el segundo comando.

Una politica que funciona en un mapa, funcionara en otro?

- 1 Entrenar con el mapa original.
- 2 Guardar el mejor individuo.
- 3 Cambiar solo los obstaculos.
- 4 Cargar la red y observar la trayectoria.
- 5 Comparar llegada, choques, pasos y ciclos.

Hipotesis

Como la red solo ve vecinos, puede reutilizar comportamientos locales, pero no conoce el mapa completo ni la distancia global a la meta.

- La clase anterior introdujo una politica que reacciona a la percepcion.
- Este laboratorio reemplaza la tabla por una red neuronal pequena.
- El algoritmo genetico sigue siendo el mecanismo de aprendizaje.
- Guardar el individuo permite separar entrenamiento y prueba.
- El mapa nuevo introduce una primera pregunta de generalizacion.

Idea final

No cambiamos el problema para ocultar la evolucion: cambiamos la representacion para estudiar que puede aprender una politica parametrica.